Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

# Department of CSE

B.Tech. CSE Trimester -5

**Roll Number:2520030011**
**Name:P.Tejaswini**

## Generative AI and AI Platforms

**25CS2207E**

# AIM

To develop an LLM Monitoring System using Python, Ollama, and MongoDB to generate AI responses and store LLM usage details in MongoDB.

# SOURCE CODE

## 1. cost_calculator.py

```
# cost_calculator.py

# Example rates per 1 million tokens.
# IMPORTANT: These are sample values, not verified current Gemini prices.
# Replace them with the actual rates for your selected model.

PRICING = {
    "gemini-3.8-flash": {
        "input": 0.50,
        "output": 2.00
    }
}


def calculate_cost(provider, model, input_tokens, output_tokens):

    # Ollama runs locally and does not charge a per-token API fee.
    if provider == "OllamaProvider":
        return 0.0

    rates = PRICING.get(model)

    if rates is None:
        return None

    input_cost = (
        input_tokens * rates["input"]
    ) / 1_000_000

    output_cost = (
        output_tokens * rates["output"]
    ) / 1_000_000

    total_cost = input_cost + output_cost

    return round(total_cost, 10)
```

## 2. llm_client.py

```
# llm_client.py

from abc import ABC, abstractmethod
from google import genai
from google.genai import types
import requests
import time

from cost_calculator import calculate_cost



# ----------------------------------------------------------------------
# 1. Common LLM Provider Interface
# ----------------------------------------------------------------------

class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass



# ----------------------------------------------------------------------
# 2. Gemini Provider
```

```
# ----------------------------------------------------------------------


class GeminiProvider(LLMProvider):


    def __init__(self, api_key):


        self.client = genai.Client(

            api_key=api_key,

            http_options=types.HttpOptions(timeout=30000)

        )


        self.model = "gemini-3-flash-pro"


    def generate(self, prompt):


        response = self.client.models.generate_content(

            model=self.model,

            contents=prompt

        )


        usage = response.usage_metadata


        input_tokens = (

            usage.prompt_token_count if usage else 0

        )
```

```
        output_tokens = (
            usage.candidates_token_count if usage else 0
        )

        return {
            "response": response.text,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens
        }



# ------------------------------------------------------------------
# 3. Ollama Provider
# ------------------------------------------------------------------

class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):

        self.model = model

        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):

        response = requests.post(
```

```
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=30
        )

        response.raise_for_status()

        data = response.json()

        return {
            "response": data["response"],
            "input_tokens": data.get("prompt_eval_count", 0),
            "output_tokens": data.get("eval_count", 0)
        }



# ----------------------------------------------------------------------
# 4. Common LLM Client
# ----------------------------------------------------------------------

class LLMClient:
```

```
def __init__(self, provider, logger):


    self.provider = provider

    self.logger = logger


def generate(self, prompt):


    max_attempts = 3


    start_time = time.perf_counter()


    last_error = None


    for attempt in range(1, max_attempts + 1):


        try:


            # Generate AI response

            result = self.provider.generate(prompt)


            # Calculate total elapsed time, including retries

            latency = round(

                (time.perf_counter() - start_time) * 1000,

                2

            )
```

```
            input_tokens = result["input_tokens"]
            output_tokens = result["output_tokens"]

            model = self.provider.model

            provider_name = type(self.provider).__name__

            cost = calculate_cost(
                provider_name,
                model,
                input_tokens,
                output_tokens
            )

            # Prepare MongoDB document
            log_data = {
                "provider": provider_name,
                "model": model,
                "prompt": prompt,
                "input_tokens": input_tokens,
                "output_tokens": output_tokens,
                "total_tokens": input_tokens + output_tokens,
                "latency_ms": latency,
                "cost": cost,
                "attempts": attempt,
                "status": "success",
```

```
                "error": None
            }

            # Log separately: a MongoDB error must not repeat
            # an already successful model request
            try:
                self.logger.log_call(log_data)
            except Exception as log_error:
                print("MongoDB logging error:", log_error)

            return result["response"]

        except Exception as error:

            last_error = error

            # Retry only likely temporary failures
            status_code = getattr(error, "status_code", None)

            retryable = (
                isinstance(
                    error,
                    (
                        requests.exceptions.Timeout,
                        requests.exceptions.ConnectionError
                    )
```

```
            )
            or status_code == 429
            or (
                isinstance(status_code, int)
                and status_code >= 500
            )
            or (
                isinstance(error, requests.exceptions.HTTPError)
                and error.response is not None
                and (
                    error.response.status_code == 429
                    or error.response.status_code >= 500
                )
            )
        )

        if not retryable or attempt == max_attempts:
            break

        wait_time = 2 ** (attempt - 1)

        print(
            f"Attempt {attempt} failed. "
            f"Retrying in {wait_time} seconds..."
        )
```

```
            time.sleep(wait_time)


    # All attempts failed, or the error was not retryable
    latency = round(
        (time.perf_counter() - start_time) * 1000,
        2
    )

    failed_log = {
        "provider": type(self.provider).__name__,
        "model": self.provider.model,
        "prompt": prompt,
        "latency_ms": latency,
        "attempts": attempt,
        "status": "failed",
        "error": str(last_error)
    }

    try:
        self.logger.log_call(failed_log)
    except Exception as log_error:
        print("MongoDB logging error:", log_error)

    raise last_error
```

### 3. mongo_logger.py

```
# mongo_logger.py

from pymongo import MongoClient
from datetime import datetime, timezone


class MongoLogger:

   def __init__(self):

       self.client = MongoClient(
           "mongodb://localhost:27017/"
       )

       self.db = self.client["llm_monitoring"]

       self.collection = self.db["call_logs"]

   def log_call(self, log_data):

       log_data["timestamp"] = datetime.now(timezone.utc)

       self.collection.insert_one(log_data)

       print("Call details saved in MongoDB!")

   def close(self):

       self.client.close()
```

### 4. main.py

```
# main.py


import os


from dotenv import load_dotenv
```

```
from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)

from mongo_logger import MongoLogger



# Load environment variables
load_dotenv(override=True)



print("================================")
print("     WEEK 3: LLM MONITORING")
print("================================")

print("1. Gemini")
print("2. Ollama")

choice = input("\nSelect provider (1/2): ").strip()

logger = None

try:
```

```
# Connect to MongoDB
logger = MongoLogger()


# Select AI provider
if choice == "1":


    api_key = os.getenv("GEMINI_API_KEY")


    if not api_key:
        raise ValueError("Gemini API key not found in .env")


    provider = GeminiProvider(api_key)


elif choice == "2":


    provider = OllamaProvider("llama3.2")


else:


    raise ValueError("Invalid selection. Choose 1 or 2.")


# Create common client
client = LLMClient(provider, logger)


# Get user prompt
prompt = input("\nEnter your question: ")
```

```
    # Generate response
    answer = client.generate(prompt)

    print("\n========== AI RESPONSE ==========")
    print(answer)

    print("\n========== COMPLETED ==========")

except Exception as error:

    print("\nApplication error:", error)

finally:

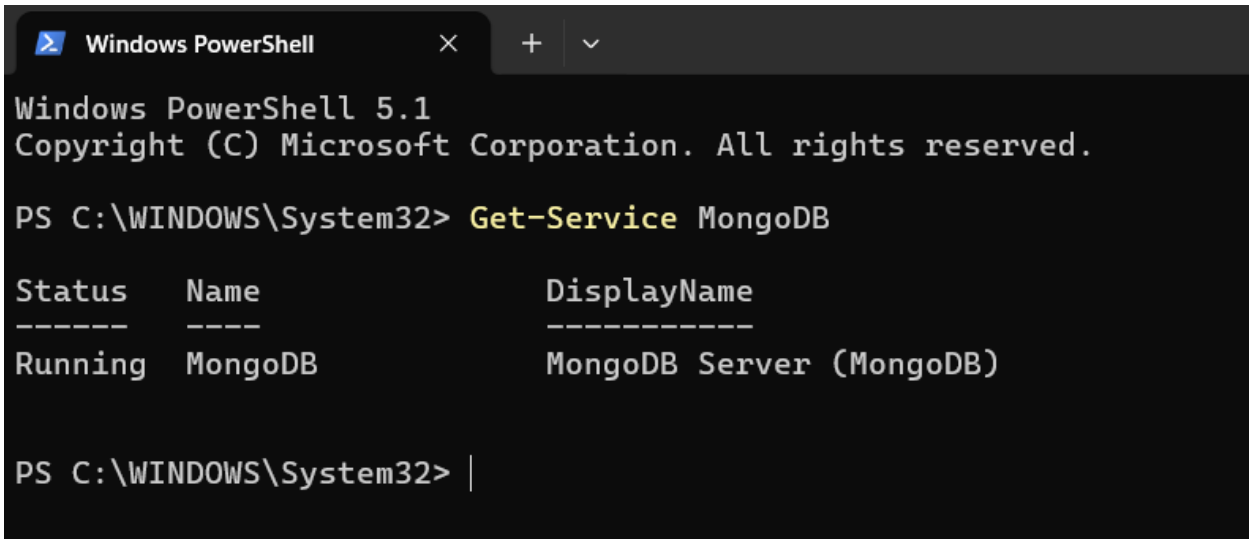
    if logger is not None:
        logger.close()
```

### 5. requirements.txt

```
pytest
google-genai
python-dotenv
pymongo
requests
```

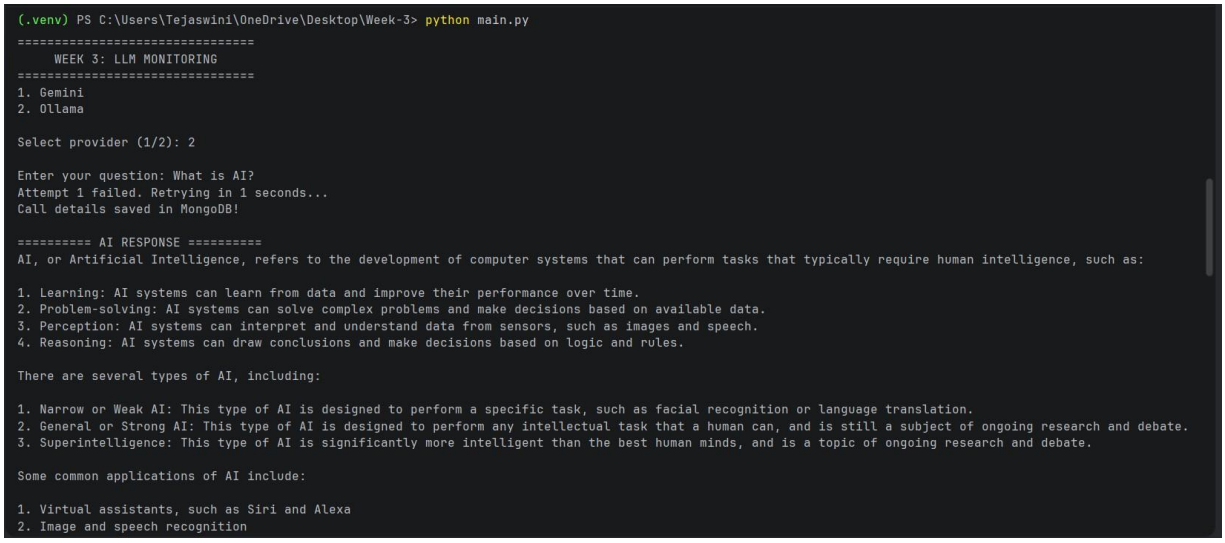
# OUTPUT

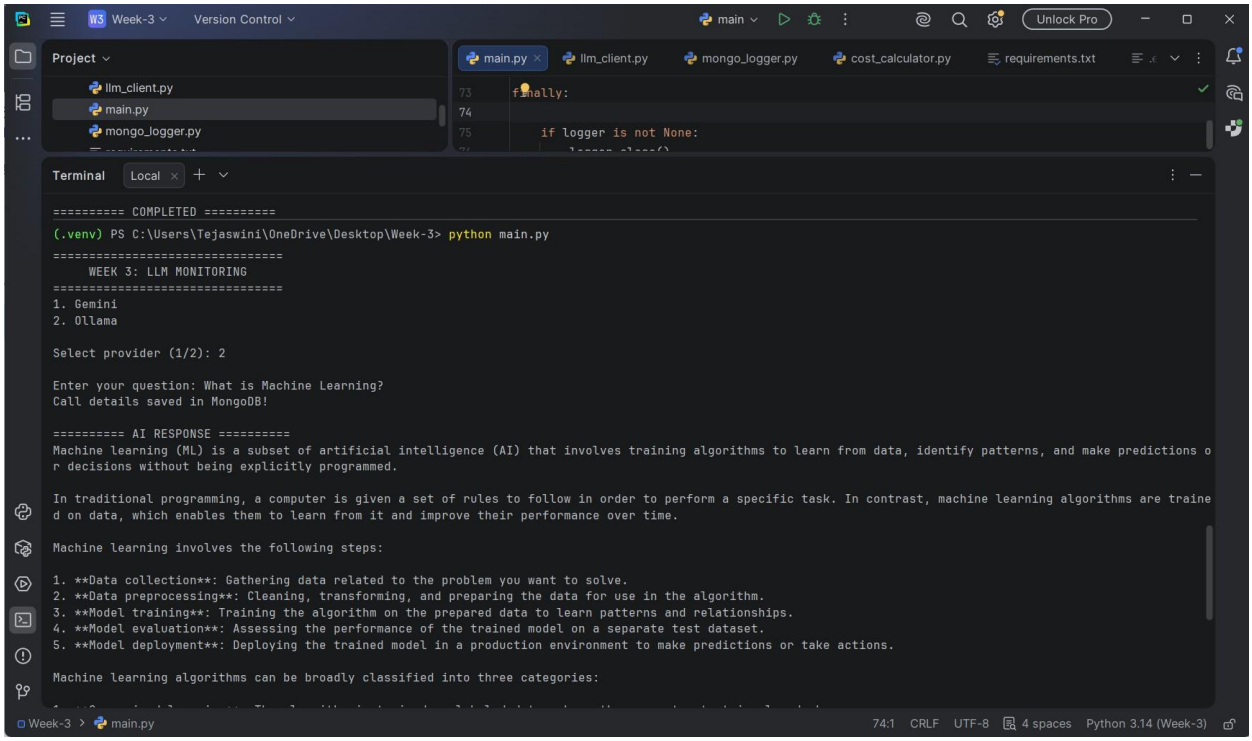
## Output 1: MongoDB Service Verification




Output: MongoDB service is Running.

## Output 2: Program Execution







```
_id: ObjectId('6af42bb713bfc01570459584')
provider : "OllamaProvider"
model : "llama3.2"
prompt : "What is AI?"
input_tokens : 29
output_tokens : 372
total_tokens : 401
latency_ms : 52047.18
cost : 0
attempts : 2
status : "success"
error : null
timestamp : ISODate('2026-11-11T08:47:51.435+00:00') November 11, 2026 at 8:47:51 AM UTC
```

```
_id: ObjectId('6af42c66ebde77f1452dd675')
provider : "OllamaProvider"
model : "llama3.2"
prompt : "What is Machine Learning?"
input_tokens : 30
output_tokens : 535
total_tokens : 565
latency_ms : 24866.5
cost : 0
attempts : 1
status : "success"
error : null
timestamp : ISODate('2026-11-11T08:50:46.308+00:00') November 11, 2026 at 8:50:46 AM UTC
```